COMPSCI 602
Project Report 1

Deva Anand
College of Information and Computer Science
University of Massachusetts
Amherst, MA 01003
devaanand@umass.edu

1 Identifying the Internal Causal Mechanism of Shallow Safety Alignment in LLMs

Phenomenon. Recent work has identified a striking behavioral pattern in safety-aligned large language models: safety constraints are concentrated in the first few generated tokens and rapidly fade over the course of generation [5]. When subjected to adversarial prefilling attacks, models that initially refuse harmful queries gradually comply as generation proceeds, suggesting that safety alignment is "shallow" encoded as a surface-level refusal pattern rather than a deep behavioral constraint. Separately, Chen et al. [2] identified a sparse set of "safety neurons" (approximately 5% of all neurons) whose activations are causally linked to refusal behavior. However, *why* the influence of these safety-critical components decays across token positions remains unexplained. This project aims to provide a causal mechanistic explanation for this decay.

System. The system under study is Llama-3.1-8B-Instruct [4], a safety-aligned large language model with 8 billion parameters. This is a pre-trained, publicly available model accessible via HuggingFace. No additional training or fine-tuning is required; all experiments are conducted at inference time using causal interventions.

Task. The task is safety-constrained response generation: given a harmful query (e.g., requesting instructions for dangerous activities), the model should produce a refusal. We study the transition from refusal to compliance as the model generates tokens beyond the initial refusal prefix.

Environment. The environment consists of adversarial prompt sets that reliably trigger shallow alignment failures. We will use two specific attack types: (1) prefilling attacks, where the model is forced to begin generation with a compliant prefix [5], and (2) adversarial suffix attacks that shift the model's behavior after an initial refusal. We vary the attack type, the number of forced prefix tokens, and the layer at which interventions are applied. Benign prompts serve as controls.

Proposed Investigation. Using the `pyvene` library [9] for interchange interventions, we will perform causal mediation analysis across layers and token positions. Specifically, we will: (a) identify which layers and attention heads carry the safety signal at the first token versus later tokens; (b) patch safety-neuron activations from early token positions into later positions to test whether restoring these activations re-triggers refusal; and (c) trace how the residual stream representation of "refuse vs. comply" evolves across token positions. This directly tests the causal mechanism behind the decay, going beyond the behavioral characterization in [5] and the neuron identification in [2].

Software and Hardware. The `pyvene` library (v1.0) provides a mature, general-purpose framework for activation interventions on PyTorch models [9]. Llama-3.1-8B-Instruct requires approximately 16 GB of GPU memory for inference, which is available on a single A100 or comparable GPU via the UMass GPU cluster. The code for the shallow alignment attacks is publicly available [10]. No model training is required.

Interest. This is my top-choice project because the shallow alignment problem has immediate practical implications for AI deployment, and solving it would represent a direct contribution to the AI safety research agenda I plan to pursue in my doctoral work.

Feasibility. This project is feasible within a semester because all experiments are inference-only interventions on a pre-trained model, with each experimental run completing in minutes on a single GPU. The tight scoping to two attack types and one model ensures the project remains manageable while producing original findings; if compute becomes a bottleneck with the 8B model, Llama-3.2-3B-Instruct serves as a viable fallback.

2 Feature Geometry of Associative Recall Across Transformers and State-Space Models

Phenomenon. Transformer and state-space model (SSM) architectures achieve similar accuracy on associative recall (AR) tasks, yet they solve them via fundamentally different internal mechanisms. Arora et al. [1] showed through interchange interventions that Transformers use a two-layer induction mechanism (storing key–value associations at value tokens), while Mamba relies on single-layer direct retrieval via short convolutions. However, *how* these different mechanisms are reflected in the geometry of the models' internal representations is entirely unknown. Do Transformers form linearly separable key–value clusters while SSMs require non-linear encoding? This project investigates the representational geometry underlying these divergent mechanisms.

System. The systems under study are small-scale Transformer and SSM architectures specifically Attention-only Transformers, Mamba, Based, DeltaNet, H3, and Hyena as implemented in the `tinylang` framework [1]. These are 2-layer models with embedding dimensions of 16–256 and millions (not billions) of parameters.

Task. The task is Associative Recall (AR): given a synthetic sequence of key–value pairs followed by a query key, the model must retrieve the corresponding value. We additionally study Associative Treecall (ATR), a hierarchical variant where keys and values are non-adjacent, generated by probabilistic context-free grammars [1].

Environment. The environment consists of synthetic formal-language datasets generated by `tinylang`, with controllable parameters: vocabulary size (up to 8192 tokens), number of key–value pairs in context, sequence length, PCFG depth and branching factor for ATR, and whether short convolutions are present or ablated in SSM architectures.

Proposed Investigation. We will characterize the representational geometry of AR solutions across architectures by: (a) training linear and non-linear probes on intermediate hidden states at the query token to test whether causal variables (key identity, value identity) are linearly separable in Transformers but not in SSMs; (b) performing PCA and representational similarity analysis (RSA) on hidden states to visualize how key–value structure is encoded; and (c) using Distributed Alignment Search (DAS) via `pyvene` [9] to identify whether the causal variables reside in low-dimensional linear subspaces. If the geometric properties differ, we will test whether intervening on the identified subspaces causally affects recall performance, connecting geometry to mechanism.

Software and Hardware. The `tinylang` library (v1.0, Stanford NLP, GitHub) provides all architecture implementations, dataset generators, training scripts, and intervention tools built on `pyvene` [9]. Models are small and train in 0.5–5 hours on a single GPU. The full experimental sweep requires only moderate compute; a focused subset for this project would need far less.

Interest. I find this project compelling because it asks a clean scientific question whether representational geometry determines computational strategy using controlled synthetic experiments that allow definitive answers.

Feasibility. This project is highly feasible within a semester because all code, datasets, and model training infrastructure are publicly available and tested, individual experiments run in minutes, and the synthetic setting provides complete control over task complexity. The main analytical tools (linear probes, PCA, DAS) are standard and well-supported by existing libraries.

3 Reverse-Engineering What ReFT Interventions Change in Model Computation

Phenomenon. Representation Finetuning (ReFT) [8] is a parameter-efficient fine-tuning method that works by learning interventions on low-rank linear subspaces of a model's hidden representations. ReFT achieves competitive or superior performance to LoRA while being 15–65$\times$ more parameter-efficient [8]. The theoretical motivation of ReFT draws on causal abstraction theory the learned interventions are intended to strengthen or redirect specific causal pathways. However, *what* these interventions actually change in the model's internal computation graph is unknown. Does ReFT achieve its efficiency by surgically targeting a small number of causally critical subspaces, or does it spread its effect more diffusely? This project aims to reverse-engineer the internal computational changes produced by ReFT.

System. The system is GPT-2 Small (124M parameters) [6], fine-tuned using LoReFT (low-rank ReFT) via the `pyreft` library [7]. GPT-2 Small is chosen because it is small enough for detailed mechanistic analysis and has extensive existing interpretability infrastructure (e.g., TransformerLens, pre-trained SAEs).

Task. The task is arithmetic reasoning on a controlled subset of GSM8K-style problems [3], where correctness is automatically verifiable without human judgment. We choose this over open-ended generation to ensure fully automated evaluation metrics (exact-match accuracy, per-step accuracy).

Environment. The environment consists of arithmetic word problems with varying difficulty levels (number of reasoning steps, operand magnitude). We apply LoReFT interventions at different layers and subspace ranks, and compare with LoRA fine-tuning under matched parameter budgets. Key experimental variables include: intervention layer, subspace rank, training data size, and problem difficulty.

Proposed Investigation. Using `pyvene` [9], we will: (a) identify which causal pathways (attention heads, MLP sublayers) are most affected by the learned ReFT interventions, by comparing activation patterns before and after ReFT at each layer; (b) perform interchange interventions to test whether the ReFT-modified subspaces are causally necessary for the improved performance i.e., does patching pre-ReFT activations into the ReFT subspace destroy the performance gain?; and (c) contrast ReFT's causal footprint with LoRA's, testing the hypothesis that ReFT concentrates changes in fewer, more causally critical locations while LoRA distributes changes more broadly. This would provide the first mechanistic account of *why* ReFT is more parameter-efficient than LoRA.

Software and Hardware. The `pyreft` library (Stanford NLP, GitHub) provides ReFT training on top of HuggingFace models [7]. The `pyvene` library handles all intervention experiments [9]. GPT-2 Small runs on a single consumer GPU. LoReFT fine-tuning on small datasets completes in under an hour. All code is open-source and actively maintained.

Interest. This project interests me because it would be the first mechanistic audit of a parameter-efficient fine-tuning method, a question I have not seen addressed anywhere in the literature.

Feasibility. This project is feasible within a semester because LoReFT fine-tuning on GPT-2 Small is computationally inexpensive (under an hour per configuration), the intervention analysis uses only forward passes, and the GSM8K evaluation is fully automated. The primary challenge is carefully designing the causal pathway comparison between ReFT and LoRA, but the existing `pyvene` API directly supports the required interchange interventions.

References

[1] Aryaman Arora, Neil Rathi, Nikil Roashan Selvam, Róbert Csórdás, Dan Jurafsky, and Christopher Potts. Mechanistic evaluation of transformers and state space models. *arXiv preprint arXiv:2505.15105*, 2025.

[2] Jianhui Chen, Xiaozhi Zheng, Zijun Yao, Yushi Bai, Lei Hou, and Juanzi Li. Towards understanding safety alignment: A mechanistic perspective from safety neurons. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John

Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[4] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aieleen Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[5] Xiangyu Qi, Ashwinee Panda, Kaifeng Lyu, Xiao Ma, Subhrajit Roy, Ahmad Beirami, Prateek Mittal, and Peter Henderson. Safety alignment should be made more than just a few tokens deep. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[6] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

[7] Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. pyReFT: Parameter-efficient fine-tuning via representations. `https://github.com/stanfordnlp/pyreft`, 2024.

[8] Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. ReFT: Representation finetuning for language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[9] Zhengxuan Wu, Atticus Geiger, Aryaman Arora, Jing Huang, Zheng Wang, Noah D. Goodman, Christopher D. Manning, and Christopher Potts. pyVene: A library for understanding and improving PyTorch models via interventions. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024.

[10] Kaifeng Lyu Xiao Ma Subhrajit Roy Ahmad Beirami Prateek Mittal Peter Henderson Xiangyu Qi, Ashwinee Panda. Shallow vs. deep alignment: Code repository. `https://github.com/Unispac/shallow-vs-deep-alignment`, 2024.